

VIISIRI Green Coding Guide (Finnish)

VIISIRI Green Coding Guide

Energiatietoinen ohjelmistokehitys eri sovellusalueilla

Tämä opas esittelee käytännöllisen ja tutkimusnäyttöön perustuvan lähestymistavan **ohjelmistojärjestelmien energialajanjäljen** pienentämiseen frontend-, backend- ja tekoälyalueilla.

Osiot

1. [Johdanto]({{ "/introduction/" | relative_url }})
2. [Vihreän koodauksen määritelmä]({{ "/defining-green-coding/" | relative_url }})
3. [Keskeiset periaatteet]({{ "/core-principles/" | relative_url }})
4. [Käytännöt sovellusalueittain]({{ "/domain-practices/" | relative_url }})
5. [Mittaaminen ja palaute]({{ "/measurement-feedback/" | relative_url }})
6. [Integrointi työnkulkuihin]({{ "/workflows/" | relative_url }})
7. [Kompromissit ja rajoitukset]({{ "/tradeoffs/" | relative_url }})
8. [Ohjelmiston resurssijälki]({{ "/footprint/" | relative_url }})
9. [Ohjelmiston kestävyyskädenjälki]({{ "/handprint/" | relative_url }})
10. [Tarkistuslistat]({{ "/checklists/" | relative_url }})
11. [Tulevaisuuden näkymät]({{ "/outlook/" | relative_url }})

1. Johdanto

Vihreä koodaus ohjelmiston laatuominaisuutena

Ohjelmistojärjestelmien energialajanjälki on muodostunut yhä tärkeämmäksi huolenaiheeksi digitaalisen infrastruktuurin jatkuvan kasvun myötä. Vaikka tieto- ja viestintätekniikan (ICT) alalla tehdyt varhaiset kestävyyspyrkimykset keskittyivät ensisijaisesti laitteistotehokkuuteen ja konesalien optimointiin, on nykyisin laajasti tunnustettu, että **ohjelmistolla itsellään on ratkaiseva rooli suorituksen aikaisen energiankulutuksen määräytymisessä**.

Ohjelmiston suunnittelu- ja toteutusvalinnat vaikuttavat suoraan siihen, kuinka intensiivisesti laitteistoresursseja — kuten CPU, muisti, verkkoliitännät, tallennuslaitteet ja kiihdyttimet — hyödynnetään. Nämä käyttömallit puolestaan määrittävät **ohjelmiston energialajanjäljen suorituksen aikana**. Jo pienet tehottomuudet koodi- tai suunnittelutasolla voivat kerryttää merkittävän energiankulutuksen, kun ohjelmistoa ajetaan laajassa mittakaavassa.

Nykyaikaiset ohjelmistojärjestelmät toimivat usein jatkuvasti, palvelevat suuria käyttäjämääriä ja perustuvat hajautettuihin arkkitehtuureihin. Tässä yhteydessä vihreä koodaus ei ole erillinen optimointitehtävä, vaan **ohjelmiston laatuun liittyvä huolenaihe**, joka on vuorovaikutuksessa suorituskyvyn, luotettavuuden, skaalautuvuuden ja ylläpidettävyyden kanssa.

Tämä opas lähestyy vihreää koodausta insinööritieteellisenä alana, joka perustuu **näyttöön ja mittaamiseen** eikä intuition tai yksittäisiin parhaiden käytäntöjen kokoelmiin. Se on kirjoitettu sekä alan ammattilaisille että tutkijoille, ja sen tavoitteena on yhdistää tutkimustulokset päivittäiseen ohjelmistokehityskäytäntöön.

2. Vihreän koodauksen määritelmä

2.1 Mitä on vihreä koodaus?

Termiä *vihreä koodaus* käytetään sekä akateemisissa että teollisuuden yhteyksissä kuvaamaan ohjelmistokehityskäytäntöjä, joiden tavoitteena on vähentää ohjelmistojärjestelmien ympäristövaikutuksia. Kirjallisuus kuitenkin osoittaa, että vihreälle koodaukselle ei ole **yhtä, yleisesti hyväksyttyä määritelmää**. Sen sijaan käsite esiintyy siihen läheisesti liittyvien termien alla, kuten *vihreä ohjelmisto*, *energiatietoinen ohjelmisto* ja *kestävä ohjelmistotuotanto* (Hilty and Aebischer, 2015;

Procaccianti et al., 2014).

Useat edustavat määritelmät havainnollistavat näkökulmien kirjoja:

- *Vihreä ja kestävä ohjelmisto* määritellään usein ohjelmistoksi, joka “minimoi kielteiset ympäristövaikutukset koko elinkaarensa aikana” (Penzenstadler et al., 2014).
- *Energiatehokas ohjelmisto* kuvataan yleisesti ohjelmistoksi, joka “vähentää energiankulutusta suorituksen aikana säilyttäen samalla hyväksyttävän suorituskyvyn” (Procaccianti et al., 2014).
- *Kestävä ohjelmistotuotanto* on määritelty tieteenalaksi, jossa ohjelmistojärjestelmiä suunnitellaan ottaen eksplisiittisesti huomioon pitkän aikavälin ympäristö-, sosiaaliset ja taloudelliset vaikutukset (Becker et al., 2016).

Nämä määritelmät eroavat toisistaan ensisijaisesti **laajuudeltaan**. Osa omaksuu laajan elinkaarinäkökulman, joka kattaa kehityksen, käyttöönoton, käytön ja käytöstäpoiston, kun taas toiset keskittyvät kapeammin **suorituksenaikaiseen käyttäytymiseen**. Tämän oppaan ensisijainen kohderyhmä koostuu ohjelmistokehittäjistä ja -arkkitehteistä, jotka tekevät konkreettisia suunnittelu- ja toteutus päätöksiä; tässä yhteydessä suoritukseen keskittyvä määritelmä tarjoaa suurimman käytännöllisen ja empiirisen arvon.

Tässä oppaassa käytetään seuraavaa työstöä helpottavaa määritelmää:

Vihreä koodaus on ohjelmiston suunnittelua ja toteutusta tavalla, joka välttää tarpeetonta suorituksenaikaista energian- ja resurssienkulutusta samalla kun toiminnalliset ja laadulliset vaatimukset täytetään.

Tämä määritelmä korostaa vihreän koodauksen useita tärkeitä ominaisuuksia:

- Ohjelmiston energialajanjälki on ensisijaisesti **suorituksenaikainen ominaisuus**, joka ilmenee ajon aikana eikä pelkästään kehitysvaiheessa.
- Energiankulutus on **kuormitusriippuvaista** ja vaihtelee syötedatan, käyttömallien ja käyttöönotto kontekstin mukaan.
- Energialajanjälkeä muovaavat **ohjelmistopäätökset**, mukaan lukien algoritmit, tietorakenteet, arkkitehtuurit, vuorovaikutusmallit ja konfigurointivalinnat.
- Energiaan liittyvät vaikutukset ovat **havaittavissa mittaamalla tai arvioimalla**, mikä mahdollistaa empiirisen arvioinnin intuition sijaan.

Keskittymällä suorituksenaikaiseen energiankulutukseen tämä määritelmä yhdistää vihreän koodauksen vakiintuneisiin ohjelmiston laadullisiin huolenaiheisiin, kuten suorituskykyyn ja skaalautuvuuteen, samalla kun se on yhteensopiva laajempien kestävyysnäkökulmien kanssa, jotka ottavat huomioon järjestelmätason ja yhteiskunnalliset vaikutukset. Vihreä koodaus soveltuu siksi useille abstraktioitasoille yksittäisistä funktioista ja metodeista kokonaisuun ohjelmistoarkkitehtuureihin.

2.2 Mitä vihreä koodaus *ei* ole

On olennaista selvittää, mitä vihreä koodaus *ei* ole, jotta voidaan välttää yleisiä väärinkäsityksiä, jotka voivat rajoittaa sen tehokkuutta tai johtaa harhautuneisiin optimointipyrkimyksiin.

Ensinnäkin vihreä koodaus **ei ole synonyymi suorituskyvyn optimoinnille**. Vaikka suoritus aika ja energiankulutus korreloivat usein keskenään, ne eivät ole toistensa vastineita. Nopeampi suoritus voi kasvattaa tehonkulutusta CPU-prosessorien tai kiihdyttimien korkeamman käyttöasteen vuoksi, mikä joissain tapauksissa johtaa suurempaan kokonaisenergiankulutukseen (Hackenberg et al., 2015). Vastaavasti hieman hitaampi suoritus saattaa vähentää energiankulutusta sallimalla laitteistokomponenttien toimia energiatehokkaammissa tiloissa. Suorituskykymittarit yksinään ovat siksi riittämättömiä energiatehokkuuden välillisiksi mittareiksi.

Toiseksi vihreä koodaus **ei ole kertaluonteinen toimenpide**. Ohjelmistojärjestelmät kehittyvät jatkuvasti ominaisuuslisäysten, refaktoroinnin, riippuvuus päivitysten ja konfigurointimuutosten myötä. Jokainen näistä muutoksista voi aiheuttaa energiaregressioita, vaikka toiminnallinen käyttäytyminen pysyisi muuttumattomana. Tästä näkökulmasta vihreä koodaus on **jatkuva huolenaihe**, joka on verrattavissa suorituskyvyn virittämiseen tai tietoturvan vahvistamiseen, eikä yksittäinen optimointivaihe.

Kolmanneksi vihreä koodaus **ei tarkoita ennenaikaista mikrooptimointia**. Eristyksissä tehdyt matalan tason optimoinnit ilman empiiristä validointia lisäävät usein koodin monimutkaisuutta tuottamatta merkityksellisiä vähennyksiä energialajanjäljessä. Kestävän ohjelmistotuotannon empiiriset tutkimukset korostavat, että merkittävimmät energiasäästöt syntyvät tyypillisesti korkean tason suunnittelu- ja arkkitehtuuripäätöksistä, jotka on validoitu mittaamalla (Procaccianti et al., 2016).

Lopuksi vihreä koodaus ei tarkoita oikeellisuuden, ylläpidettävyyden tai muiden

laadullisten ominaisuuksien oletusarvoista uhraamista. Sen sijaan se edistää **eksplisiittisiä ja harkittuja kompromisseja**, joissa energiatehokkuus otetaan huomioon toiminnallisten ja ei-toiminnallisten vaatimusten rinnalla. Tämä näkökulma asettaa vihreän koodauksen vastuullisen ohjelmistotuotannon olennaiseksi osaksi pikemminkin kuin valinnaiseksi tai toissijaiseksi huolenaiheeksi.

Viitteet

Becker, C., Betz, S., Chitchyan, R., Duboc, L., Easterbrook, S.M., Penzenstadler, B., Seyff, N. and Venters, C.C. (2016) *Requirements: The key to sustainability*. IEEE Software, 33(1), pp.56–65.

Hackenberg, D., Schöne, R., Ilsche, T., Molka, D., Schuchart, J. and Geyer, R. (2015) *An energy efficiency feature survey of the Intel Haswell processor*. IEEE International Parallel and Distributed Processing Symposium Workshops, pp.896–904.

Hilty, L.M. and Aebischer, B. (2015) *ICT for sustainability: An emerging research field*. In: Hilty, L.M. and Aebischer, B. (eds.) *ICT Innovations for Sustainability*. Springer, pp.3–36.

Penzenstadler, B., Bauer, V., Calero, C. and Franch, X. (2014) *Sustainability in software engineering: A systematic literature review*. Proceedings of the 16th International Conference on Evaluation & Assessment in Software Engineering (EASE), pp.1–10.

Procaccianti, G., Lago, P. and Vetro', A. (2014) *Energy efficiency in software: A systematic literature review*. Journal of Systems and Software, 97, pp.135–155.

Procaccianti, G., Lago, P. and Lewis, G. (2016) *Green architectural tactics for the cloud*. Proceedings of the 2016 IEEE/ACM International Conference on Software Architecture (ICSA), pp.41–50.

3. Vihreän koodauksen keskeiset periaatteet

Seuraavat periaatteet muodostavat teknologiariippumattoman perustan ohjelmistojärjestelmien suorituksenaikaisen energiatilan pienentämiselle. Ne soveltuvat kaikille aloille ja abstraktiotasolle yksittäisistä funktioista hajautettuihin arkkitehtuureihin. Näitä periaatteita toteuttavat toimialakohtaiset käytännöt on kuvattu [luvussa 4](#).

3.1 Vältä tarpeetonta työtä

Tehokkain tapa vähentää energiankulutusta on välttää laskentaa, tietojenkäsittelyä ja viestintää, joilla ei ole toiminnallista arvoa. Tähän kuuluvat mm. logiikka, joka suoritetaan jokaisen pyynnön yhteydessä mutta on merkityksellinen vain harvoin, data, joka haetaan mutta jota ei koskaan käytetä, sekä operaatiot, jotka käynnistyvät tapahtumista, jotka eivät niitä edellytä.

Tarpeeton työ on usein näkymätöntä, koska se jakautuu moniin pieniin operaatioihin, joista kukin vaikuttaa yksittäin harvittomalta. Kumulatiivinen vaikutus useiden käyttäjien, pyyntöjen tai laitteiden välillä voi kuitenkin olla huomattava. Sen tunnistaminen edellyttää profilointia pelkän tarkastelun sijaan, sillä kallein koodi ei ole harvoin ilmeisin.

Tämä periaate koskee myös arkkitehtuuritasoa. Käsittely, jota voitaisiin välttää paremmalla järjestelmäsuunnittelulla — kuten saman tuloksen toistuvalla uudelleenlaskemisella tietokannasta — edustaa tarpeetonta työtä riippumatta siitä, kuinka tehokkaasti se suoritetaan.

3.2 Vältä toistuvaa työtä

Saman työn toistuva suorittaminen kasvattaa kumulatiivista energiatilaa lisäämättä toiminnallista arvoa. Uudelleenkäyttö, välimuistitus ja muistointi voivat merkittävästi vähentää redundanttia resurssien käyttöä, kun niitä sovelletaan harkitusti ja tuloksia mitataan huolellisesti. Empiirinen arviointi API-suunnitteluvaihtoelista on osoittanut, että muistinsisäisen välimuistituksen käyttöönotto toistuviin kyselyihin voi vähentää tehonkulutusta noin 15 %, kun taas harkittoman algoritmisen logiikan korvaaminen optimoiduilla vastineilla voi tuottaa jopa 23 %:n säästöjä (Joof et al., 2025; Joof, 2025).

Oleellinen varaus on *harkitusti ja mitaten*. Välimuistitus ei ole yleisesti hyödyllistä: se kuluttaa muistia, aiheuttaa vanhentuneen datan riskin, ja säästää energiaa vain

silloin, kun osumataajuus on riittävän korkea kattaakseen lisäkuorman. Sama koskee esikäsitteilyä, joka siirtää työn pyyntöajasta rakennus- tai käynnistysaikaan. Tämä saattaa vähentää pyyntikohtaista energiaa, mutta lisää järjestelmätason energiaa, jos esikäsitellyt tulokset ovat harvoin käytössä.

Toistuva työ on erityisen yleinen ongelma hajautetuissa järjestelmissä, joissa useat komponentit hakevat saman datan itsenäisesti, sekä frontend-sovelluksissa, joissa komponentit renderöityvät uudelleen tilaan muutoksiin vastatessa, vaikka muutos ei vaikuta niihin.

3.3 Siirrä ja tallenna vähemmän dataa

Datan siirto ja tallennus kuluttavat merkittävästi energiaa erityisesti hajautetuissa järjestelmissä ja mobiiliympäristöissä. Datan siirtäminen verkon yli, levyille kirjoittaminen ja sieltä lukeminen sekä datan kopiointi muistialueiden välillä kuluttavat energiaa siirrettyyn volyymiin suhteutettuna.

Hyötykuormien koon pienentäminen, redundantin serialisoinnin ja deserialisoinnin poistaminen, datan pakkaaminen silloin kun pakkauksen CPU-kustannus on pienempi kuin pienemmästä siirrosta saatava energiasäästö, sekä tarpeettoman pysyvyytalletuksen välttäminen ovat kaikki tämän periaatteen käytännön ilmentymiä.

Periaate soveltuu kaikissa mittakaavoissa: tietokantakyselyssä valittujen sarakkeiden määrän vähentäminen, API:n palauttamien kenttien karsiminen, lokitulosteen suodattaminen ja käyttämättömien riippuvuuksien poistaminen ohjelmistoniputuksesta ovat kaikki esimerkkejä vähemmän datan siirtämisestä ja tallentamisesta.

3.4 Suosi tehokkaita abstraktioita

Abstraktiot parantavat ylläpidettävyyttä, mutta voivat piilottaa kalliita operaatioita. Kätevä metodikutsu, korkean tason kehysominaisuus tai laajasti käytetty kirjasto saattaa peittää merkittävän laskennallisen lisäkuorman, erityisesti kun sitä käytetään kuumassa polulla, suuressa mittakaavassa tai resurssirajoitteisessa ympäristössä.

Vihreä koodaus kannustaa tietoisuuteen käytössä olevien abstraktioiden resurssi- ja energiavaikutuksista. Tämä ei tarkoita abstraktioista luopumista matalan tason koodin hyväksi, sillä useimmissa tapauksissa siitä aiheutuva ylläpidettävyyden kustannus ylittää selvästi energiahyödyn. Se tarkoittaa harkitsevuutta: ymmärrystä siitä, mitä abstraktio tekee kulissien takana, kevyempien vaihtoehtojen valitsemista silloin kun ne ovat toiminnallisesti vastaavia, ja profilointia sen sijaan, että oletetaan tutun työkalun olevan tehokas.

Tämä periaate on erityisen merkityksellinen kehyksiä, suoritusympäristöjä, serialisointiformaatteja ja viestintäprotokollia valittaessa, sillä valinta vaikuttaa koko järjestelmään yksittäisen operaation sijaan.

3.5 Tee energiajälki näkyväksi

Ilman mittausta energiatehokkuus jää spekulatiiviseksi. Kehittäjät eivät pysty luotettavasti tunnistamaan järjestelmän energiantensiivisimpiä osia pelkän tarkastelun avulla, eikä energiaparannuksia voida todentaa ilman vertailuperustaa.

Näkyvyys profiloinnin ja vertailun kautta on välttämätöntä näyttöön perustuvalle vihreälle koodaukselle. Tämä tarkoittaa energia- tai resurssilähtötasojen asettamista ennen optimointia, muutosten vaikutuksen mittaamista oletusten sijaan, sekä energiaan liittyvän tiedon saattamista tiimin saataville koontinäyttöjen, CI-mittareiden tai koodikatselmusartefaktien kautta, jotta se voi ohjata päätöksiä pitkällä aikavälillä.

Näkyvyys tarkoittaa myös epävarmuuden myöntämistä. Energianmittaustyökaluilla on rajoituksensa (ks. luku 7), ja tuloksia tulisi tulkita vertailevasti ja läpinäkyvästi eikä täsmällisinä absoluuttisina arvoina. Prototyypitutkimus, jossa energiaprofilointi integroitiin suoraan versionhallinnan työnkulkuihin, on osoittanut, että commit-tason energiapalaute lisää kehittäjien tietoisuutta kestävydestä ja vaikuttaa suunnittelupäätöksiin — jopa niiden kehittäjien kohdalla, jotka eivät aiemmin olleet pitäneet energiaa laatutekijänä (Joof et al., 2025; Joof, 2025).

Viitteet

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta-Lahti

4. Vihreän koodauksen käytännöt toimialoittain

Vihreän koodauksen periaatteet soveltuvat kaikille toimialoille, mutta niiden käytännöllinen vaikutus riippuu suorituskontekstista. Tässä osiossa käytännöt on järjestetty toimialoittain keskittyen **suorituksenaikaisen energialajanjäljen keskeisiin tekijöihin**. Alla esitetyt käytännöt laajentavat [osion 3](#) ydinyperiaatteita ja tarjoavat käytännön lähtökohdan kullekin toimialalle.

4.1 Frontend – Verkkosovellukset

Energialajanjäljen keskeisiä tekijöitä ovat JavaScript-suoritus, renderöinti ja verkkoliikenne. Koska sama koodi suoritetaan miljoonilla käyttäjien laitteilla, frontendin tehottomuus moninkertaistuu koko käyttäjäkunnan laajuudella, jolloin jopa pienet sivukohtaiset parannukset ovat merkittäviä laajassa mittakaavassa (Manotas et al., 2016).

Minimoi tarpeettomat uudelleenrenderöinnit ja layout thrashing

Komponenttipohjaisissa kehyksissä vanhemman tilan muutos voi käynnistää uudelleenrenderöinnin alipuissa, joihin muutos ei vaikuta. Jokainen uudelleenrenderöinti sisältää erojen laskemisen, DOM:n paikkaamisen, tyylin uudelleenlaskennan ja mahdollisesti layoutin, jotka kaikki ovat CPU-työtä, joka tuottaa lämpöä käyttäjän laitteessa. Käytä muistintallennusta (memoisation), valitsijafunktioita ja komponenttien pilkkomista varmistaaksesi, että vain todella muuttuneet komponentit päivittyvät. Vältä myös DOM-lukujen ja -kirjoitusten vuorottelua JavaScript-silmukoissa, mikä pakottaa selaimen laskemaan layoutin toistuvasti uudelleen (tunnetaan nimellä layout thrashing).

Pienennä bundle-kokoa koodin jakamisella ja laiskalla latauksella Suuremmat JavaScript-bundlet vaativat enemmän jäsentämistä, kääntämistä ja suoritusaikaa, joista kaikki kuluttavat energiaa ennen kuin yhtäkään sovelluksen logiikan riviä ajetaan. Koodin jakaminen varmistaa, että käyttäjät lataavat ja suorittavat vain sen koodin, jota tarvitaan heidän nykyisessä näkymässään. Laiska lataus lykkää muun lataamista siihen asti, kun sitä todella tarvitaan. Tarkasta bundlen koostumus säännöllisesti rakennusanalyysityökaluilla ja poista käyttämättömät riippuvuudet.

Vältä runsasta API-liikennettä Useat pienet, peräkkäiset API-kutsut pitävät verkkoliikennän aktiivisena ja lisäävät viivettä. Jokaisella verkkopyynnöllä on myös kiinteä energiakustannus hyötykuorman koosta riippumatta: yhteyden muodostaminen, DNS-selvitys ja TCP/TLS-käyttelyn suorittaminen. Yhdistä toisiinsa liittyvät pyynnöt, käytä HTTP/2-multipleksointia tarvittaessa ja harkitse kyselykieliä (kuten GraphQL), joilla asiakas voi määrittää tarvitsemansa tiedot tarkasti.

Käytä välimuistia ja HTTP-mekanismeja tehokkaasti Tarpeeton tiedonsiirto on yksi eniten vältettävistä frontendin energiankulutuksen lähteistä. Cache-Control-otsikoiden, ETagien ja service workereiden oikea käyttö voi poistaa kokonaan muuttamattomien resurssien uudelleenlataukset. Mittaa todelliset välimuistin osumataajuudet varmistaaksesi, että välimuisti toimii tarkoitetulla tavalla.

Suosi CSS- ja laitteistokiihdytettyjä animaatioita JavaScript-silmukoiden sijaan JavaScript-ohjatut animaatiot toimivat pääsääntöisesti ja kilpailevat layoutin ja renderöinnin kanssa. CSS-siirtymät ja Web Animations API delegoivat työn compositor-säikeelle, joka on tyypillisesti GPU-kiihdytetty eikä estä pääsääntöisen suoritusta. Käytä animaatioissa transform- ja opacity-ominaisuuksia, sillä nämä ominaisuudet voidaan yhdistää ilman layoutin tai piirron käynnistämistä.

4.2 Frontend – Mobiilisovellukset

Mobiilin energialajanjälki koostuu pääasiassa akkujen käytöstä CPU:n, verkkoradioiden, näytön ja antureiden toiminnassa. Laitteen herättäminen matalan virrankulutuksen tilasta on merkittävä kustannus: yksi tarpeeton herätys voi kuluttaa yhtä paljon energiaa kuin monta millisekuntia normaalia CPU-toimintaa (Pathak et al., 2012).

Vähennä taustatoimintaa ja herätyksiä Jokainen taustaherätys pakottaa CPU:n ja mahdollisesti verkkoradioiden pois matalan virrankulutuksen unitilasta. Yhdistä

taustatoiminnot alustan tarjoamia API:ja käyttäen, kuten WorkManager Androidilla ja BGTaskScheduler iOS:llä, jotka sallivat käyttöjärjestelmän ajoittaa työn laitteen jo aktiivisina tai latauksessa olevina jaksoina. Vältä erillisiä ajastimia ja hälytyksiä tehtäville, jotka sietävät eräajoa.

Niputa verkkopyynnöt ja tue offline-työnkulkua Matkapuhelinradio (LTE/5G) on yksi mobiililaitteen energiasäästöisimmistä komponenteista ja kuluttaa merkittävästi virtaa myös siirron päätyttyä olevan odotusjakson aikana. Pyyntöjen niputtaminen harvempiin, suurempiin vuorovaikutuksiin lyhentää aikaa, jonka radio viettää aktiivisessa ja odotustilassa. Offline-työnkulkujen tukeminen tallentamalla ja toistamalla toiminnot, kun yhteys palautuu, vähentää edelleen tarpeetonta tiedonsiirtoa.

Optimoi renderöinti ja vierittäminen Kuvaruudun ulkopuolinen renderöinti, liiallinen overdraw (saman pikselin piirtäminen useita kertoja per kuva) ja toistuvat layoutin läpikäynnit kuluttavat akkua suoraan. Käytä näkymäkierrätysmenetelmiä (esim. RecyclerView, LazyColumn) välttääksesi näytön ulkopuolisen sisällön renderöinnin. Vähennä overdrawn tasaistaen näkymähierarkioita ja poistaen etusisällön peittämät opaakit taustat.

Käytä antureita säästeliäästi ja sopivalla tarkkuudella GPS täydellä tarkkuudella ja korkealla kyselytaajuuksella on yksi mobiililaitteen energiasäästöisimmistä antureista. Jos karkea sijainti riittää, käytä `PRIORITY_LOW_POWER`- tai geofencing-menetelmää jatkuvan tarkan seurannan sijaan. Sovella samaa periaatetta kiihtyvyysantureihin, gyroskooppeihin ja mikrofoneihin: pyydä alin näytteenottotaajuus ja tarkkuus, joka täyttää toiminnallisen vaatimuksen, ja poista kuuntelijoiden rekisteröinti, kun niitä ei enää tarvita.

4.3 Backend – Palvelut ja API:t

Backendin energiajalanjälkeä ohjaavat CPU-käyttö per pyyntö, tietojenkäyttömallit ja verkkoviestinnän volyymi. Koska yksi backend-instanssi palvelee monia samanaikaisia käyttäjiä, tehottomuudet kertautuvat nopeasti liikenteen kasvaessa.

Vähennä pyyntökohtaista työtä Profiloivat kuumat polut tunnistaaksesi operaatiot, jotka suoritetaan jokaisella pyynnöllä. Jopa yhden kalliin tietokantakutsun, laskennan tai ulkoisen palvelukutsun poistaminen tai lykkääminen vilkkaasta päätepuolesta voi tuottaa merkittäviä kokonaisenergian säästöjä palvelinpuolella. Kohteile pyyntökohtaista CPU-aikaa resurssipankkina, jota tulee puolustaa koodikannan kehittyessä. Hallitussa tutkimuksessa, jossa verrattiin API-suunnitteluvaihtoehtoja, $O(n^2)$ -sisäkkäissilmukkalajittelun korvaaminen optimoidulla sisäänrakennetulla vastineella vähensi tehonkulutusta noin 23 % ja vasteaikaa 43 % optimoimattomaan lähtötasoon verrattuna (Joof et al., 2025; Joof, 2025).

Vältä N+1-kyselyitä ja liiallista hakemista Entiteettilistan lataaminen ja sen jälkeen yksittäisten kyselyiden tekeminen jokaiselle entiteetille on yksi yleisimmistä ja vältettävissä olevista energiantuhlauksen malleista backend-kehityksessä. Korvaa N+1-mallit yksittäisellä erä-kyselyllä tai ORM:n tai kyselynrakentajan tarjoamalla innokkaaseen lataamiseen tarkoitetulla mekanismilla. Yhtä tärkeää on valita vain tarvittavat sarakkeet, sillä kokonaisten rivien hakeminen, kun tarvitaan vain kaksi kenttää, tuhlaa I/O-kaistanleveyttä ja lisää serialisointiliikkuuria.

Käytä rajattua, mitattua välimuistia Välimuisti on hyödyllinen vain, jos osumataajuus oikeuttaa muistijalanjäljen ja vanhenemisriskin. Rajoittamaton välimuisti lisää GC-painetta ja muistinkulutusta; väärällä poistokäytännöllä varustettu välimuisti saattaa pitää sisällään vanhentuneita tai harvoin käytettyjä merkintöjä. Instrumentoi välimuistit osumataajuuksien mittaamiseen ja aseta TTL:t ja kokorajoitukset havaittuihin käyttömalleihin eikä oletusarvoihin perustuen. Empiiriset tulokset API-tason profiloinnista osoittavat, että muistinsisäisen välimuistin käyttöönotto toistuviin kyselyihin vähentää tehonkulutusta noin 15 % ja vasteaikaa 36 % välimuistittomaan lähtötasoon verrattuna (Joof et al., 2025; Joof, 2025).

Minimoi hyötykuorman koko ja serialisointiliikkuuria JSON-serialisointi ja -deserialisointi on CPU-intensiivistä suuressa mittakaavassa. Palveluiden välisessä sisäisessä viestinnässä binäärimuodot kuten Protocol Buffers tai MessagePack voivat vähentää sekä CPU-aikaa että siirtomäärää. Ulkoisille API:lle tarjoa vastauksen kenttäsuodatus (esim. fields-kyselyparametrien tai GraphQL-valintojen kautta), jotta asiakkaita ei pakoteta vastaanottamaan ja hylkäämään tietoa, jota he eivät käytä. Huomaa, että hyötykuorman pakkaaminen sisältää selkeän kompromissin: GZIP-koodaus vähentää verkonsiirtoa mutta lisää CPU-kulutusta, ja nettoenergian vaikutus riippuu siirron ja käsittelyn suhteellisista kustannuksista käyttöympäristössä (Joof et al., 2025; Joof, 2025).

4.4 Backend – Datat käsittely ja tallennus

Dataputket kuluttavat energiaa pitkittyneen suorituksen ja laajamittaisen I/O:n

kautta. Toisin kuin interaktiiviset palvelut, putkilinjat toimivat usein ilman vastaavan käyttäjän odottamista, mikä tarkoittaa, että niiden energiankulutus on vähemmän näkyvää mutta ei yhtään vähäisempää.

Vältä muuttumattoman datan täydellistä uudelleenlaskentaa Putkilinjan uudelleenajaminen syötedatalle, joka ei ole muuttunut edellisestä ajosta, on puhdasta tuhlausta. Tunnista muuttumattomat osiot tarkistussummien, muokkausaikaleimamerkintöjen tai inkrementaalisten laskentakehysten (kuten Apache Sparkin structured streaming tai dbt:n inkrementaalimallit) avulla ja ohita ne. Jopa osittainen uudelleenlaskennan laajuuden vähennys voi merkittävästi pienentää putkilinjan kokonaisenergiaa.

Käytä inkrementaalista ja tarkistuspiestepohjaista käsittelyä Pitkään suorittavat eräajot, jotka käynnistyvät alusta epäonnistumisen jälkeen, tuhlaavat kaiken epäonnistuneeseen ajoon investoidun energian. Tarkistuspiesteytys, joka tallentaa ajoittain välivaiheen tilan kestäväään tallennustilaan, mahdollistaa putkilinjan jatkamisen viimeisestä vakaaasta kohdasta alun sijaan. Tarkistuspiestien kirjoittamisen energiakustannus on tyypillisesti pieni suhteessa täyden uudelleenkäynnistykseen kustannukseen.

Sovella datan säilytyskäytäntöjä ja elinkaaripolitiikkoja Tallennusenergia kasvaa datamäärän mukana: enemmän dataa tarkoittaa enemmän levytiloja, enemmän indeksien ylläpitoa sekä enemmän varmuuskopiointi- ja replikointiylivuoraa. Määrittele ja noudata säilytyskäytäntöjä, jotka poistavat datan, kun sitä ei enää tarvita operatiivisiin, analyttisiin tai vaatimustenmukaisuustarkoituksiin. Siirrä kylmä data, kuten harvoin kysely historiallinen tieto, pienitehoisempiin tallennuskerroksiin sen sijaan, että se pidetään suorituskykyisessä pääasiallisessa tallennuksessa.

Sovita käsittelytaajuus todellisiin tarpeisiin Tunnin välein ajettavan putkilinjan suorittaminen tulosten tuottamiseksi, joita kulutetaan päivittäin, on tarpeetonta työtä. Tarkista kunkin putkilinjan aikataulutustiheys suhteessa alajuoksun kuluttajien todelliseen päätöksen viivevaatimukseen. Kun viive on hyväksyttävissä, suosi tapahtumaohjattuja käynnistimiä kiinteiden aikataulujen sijaan välttääksesi tyhjiä tai minimaalisten lisäysten käsittelyn.

4.5 Backend – Infrastrukturi ja pilvi

Infrastruktuurivalinnat määrittävät ohjelmiston ajamisen perusenergian ennen kuin yhtään pyyntöä on palveltu. Yliprovisoidut tai huonosti konfiguroidut infrastruktuurit tuhlaa energiaa jatkuvasti, ei pelkästään kuormituksen aikana.

Mitoita laskentaresurssit oikein Instanssi, joka toimii 5 % keskimääräisellä CPU-käyttöasteella, kuluttaa suunnilleen saman verran joutokäyntitehoa kuin instanssi 60 % käyttöasteella, tuottaen samalla huomattavasti vähemmän hyödyllistä työtä per watti. Profiloitu todellinen resurssienkäyttö edustavien kuormitusjaksojen aikana ja mitoita instanssit sen mukaisesti. Pilviympäristöt tekevät tästä palautettavan toimenpiteen, joten aloita konservatiivisesti ja skaalaa havaitun kysynnän eikä teoreettisten huippujen perusteella.

Käytä nollaan skaalautuvia ja palvelimettomia malleja purskuvaisille kuormituksille Palvelut, jotka ovat joutokäynnillä merkittävän osan päivästä, hyötyvät arkkitehtuurista, jotka mahdollistavat laskentaresurssien skaalaamisen nollaan käyttämättömänä aikana. Palvelimettomat funktiot, konttipohjainen automaattinen skaalaus (esim. Knative, AWS Lambda, Google Cloud Run) ja tuotantoympäristöjen ulkopuolisten ympäristöjen ajoitettu sammuttaminen vähentävät kaikki energiankulutusta alhaisen kysynnän jaksoina.

Harkitse käyttöönottoalueiden hiili-intensiteettiä Sama kuormitus tuottaa erilaiset elinkaarihiilidioksidipäästöt riippuen siitä, missä se ajetaan. Pääosin uusiutuvilla energialähteillä toimivilla pilvipalvelualueilla on huomattavasti alhaisempi hiili-intensiteetti kuin kivihiiheen tai kaasuun nojaavilla alueilla. Kuormituksille, joissa viivevaatimukset sallivat maantieteellisen joustavuuden, kuten eräajoille, asynkroniselle käsittelylle tai dataputkille, suosi alueita, joilla on alhaisempi hiili-intensiteetti. Useimmat suuret pilvipalveluntarjoajat julkaisevat reaaliaikaisia tai historiallisia hiili-intensiteettitietoja alueidensa osalta.

Pakota automaattinen skaalaus ja vältä pitkittynyttä joutilaana yliprovisiointia Kiinteäkapasiteettisiin käyttöönottoihin, jotka on mitoitettu huippukuormitusta varten, jää täysi kapasiteetti varattuna ja energiaa kuluttavana hiljaisina tunteina. Konfiguroi automaattiset skaalausikäytännöt, jotka vähentävät replikoiden määrää, instanssikokoa tai klusterin solmumäärää pitkittyneiden matalan liikenteen jaksojen aikana. Yhdistä tämä ajoitettuun skaalaukseen ennustettaville liikennemalleille (esim. yön yli skaalaaminen alas palveluille, joilla on tunnettu vuorokautinen käyttömalli).

4.6 Tekoälyn kehittäminen – Kouluttaminen

Tekoälyn koulutuksen energialajanjälkeä dominoi kiihdyttimen (GPU/TPU) suoritus aika. Suuren mallin kouluttaminen voi kuluttaa satoja tai jopa tuhansia kilowattitunteja, mikä usein ylittää monen käyttöönoton elinikäisen päättelykäytön energian (Strubell et al., 2019; Patterson et al., 2021).

Mitoita mallit ja aineistot oikein Suuremmat mallit ja aineistot eivät aina tuota suhteessa parempia tuloksia tiettyä tehtävää varten. Arvioi, täyttääkö pienempi, hienosäädetty tai distilloitu malli laatuvaatimukset ennen laajamittaisen koulutuksen aloittamista. Tarkasta myös koulutusaineistot duplikaattien ja huonolaatuisten näytteiden varalta, jotka lisäävät laskenta-aikaa parantamatta yleistämiskykyä.

Vältä tarpeettomat kokeet Koulutusajot lähes identtisillä hyperparametreilla tai keskeytyneen kokeen uudelleenajaminen ilman tarkistuspisteen tallentamista ovat yleisiä vältettävissä olevan energiankulutuksen lähteitä. Käytä kokeenseurantatyökaluja (kuten MLflow tai Weights & Biases) konfiguraatioiden ja tulosten kirjaamiseen ja aseta konvergensskriteerit ennen ajon käynnistämistä eikä jälkikäteen arvioiden.

Sovella varhaista pysäyttämistä ja tehokkaita koulutustekniikkoja Seuraa validointimittareita koulutuksen aikana ja pysähdy, kun parannus tasaantuu, sen sijaan että ajettaisiin kiinteä määrä epocheja. Sekatarkkuuskoulutus (FP16 tai BF16) vähentää muistikaistanleveyskulutusta ja laskenta-aikaa per vaihe tuetuilla laitteistoilla tyypillisesti ilman merkittävää laadunylivuormaa. Oppimistahdin aikataulutus, gradienttien akkumulointi ja tehokkaat huomio-toteutukset ovat edelleen vakiintuneita tekniikoita kokonaislaskennan vähentämiseen.

Maksimoi laitteiston käyttöaste Matala GPU-käyttöaste koulutuksen aikana, joka johtuu yleisesti datan latauksen pullonkauloista, CPU:n esikäsittelystä tai tehoton erästyksen, tarkoittaa, että energiaa kulutetaan ilman tuottavaa työtä. Profilo GPU-käyttöaste ja korjaa putkilinjan hidastukset esihakemalla dataa, lisäämällä datan latausohjainten määrää tai esikäsittelemällä ja tallentamalla syötteet välimuistiin offline-tilassa.

4.7 Tekoälyn käyttö – Päättely

Päättelyn energialajanjälki kasvaa pyyntömäärän mukana. Toisin kuin koulutus, joka tapahtuu kerran malliversiota kohti, päättely toimii jatkuvasti tuotannossa ja sen kumulatiivinen energiakustannus ylittää usein koulutuskustannuksen mallin käyttöönottokauden aikana.

Käytä pienintä tehokasta mallia Mallin monimutkaisuus tulisi sovittaa tehtävävaatimuksiin. Suuri yleiskäyttöinen malli, jota sovelletaan kapeaan, hyvin määritellyyn tehtävään, on todennäköisesti merkittävästi ylimitoitettu. Tekniikat kuten kvantisointi (numeerisen tarkkuuden pienentäminen), karsinta (pieniarvoisien yhteyksien poistaminen) ja distillaatio (pienemmän mallin kouluttaminen replikoimaan suuremman mallin käyttäytymistä) voivat merkittävästi vähentää päättelyenergiaa ilman suhteellista häviötä tulostenlaadussa.

Minimoi kehotteiden ja syötteiden koko Transformer-pohjaisissa malleissa itseohjautuvan huomion monimutkaisuus kasvaa neliöllisesti syötteiden pituuden mukaan. Pidemmät kehotteet ja suuremmat kontekstiikkunat lisäävät suoraan pyyntökohtaista laskentakustannusta. Karsii epäolennainen konteksti, vältä monisanaisia kehotemallipohjia ja arvioi, voidaanko haetut kontekstikatkelmia lyhentää vaikuttamatta vastauslaatu.

Tallenna deterministiset tulosteet ja upotukset välimuistiin Identtiset tai semanttisesti vastaavat kyselyt, jotka tuottavat luotettavasti saman tulosteen, tulisi palvella välimuistista eikä käynnistää uudelleenpäättelyä. Kiinteiden viitedokumenttien upotuslaskelmat tulisi vastaavasti esihakea ja tallentaa välimuistiin eikä laskea uudelleen jokaisella pyynnöllä. Semanttinen välimuistitus, joka täsmää toistensa parafraseja olevat kyselyt, voi ulottaa välimuistin osumataajuudet tarkkojen vastaavuuksien ulkopuolelle.

Niputa pyynnöt mahdollisuuksien mukaan Erästys jakaa mallipainojen muistista lataamisen kiinteän kustannuksen ja mahdollistaa kiihdyttimen toimimisen korkeammalla käyttöasteella. Asynkronisille tai lähes reaaliaikaisille käyttötapauksille kerää pyynnöt eriin ja käsittele ne yhdessä. Dynaaminen erästys, joka ryhmittelee lyhyen aikavälin sisällä saapuvat pyynnöt, voi saavuttaa suurimman osan läpäisykykyhyödyistä ilman merkittävää lisäviivettä.

Viitteet

Manotas, I., Bird, C., Zhang, R., Shepherd, D., Jaspan, C., Sadowski, C., Pollock, L. and Clause, J. (2016) *An empirical study of practitioners' perspectives on green software engineering*. Proceedings of the 38th International Conference on Software Engineering (ICSE), pp.237–248.

Pathak, A., Hu, Y.C. and Zhang, M. (2012) *Where is the energy spent inside my app? Fine-grained energy accounting on smartphones with Eprof*. Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys), pp.29–42.

Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.M., Rothchild, D., So, D., Texier, M. and Dean, J. (2021) *Carbon emissions and large neural network training*. arXiv preprint arXiv:2104.10350.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J.P. and Saraiva, J. (2017) *Energy efficiency across programming languages: How do energy, time, and memory relate?* Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering, pp.256–267.

Strubell, E., Ganesh, A. and McCallum, A. (2019) *Energy and policy considerations for deep learning in NLP*. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp.3645–3650.

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

5. Mittaaminen ja palaute

UTU täyttää tämän osion

6. Vihreän koodauksen integroiminen kehitystyönkulkuun

Vihreä koodaus on kestävä käytäntö vain silloin, kun se sopii olemassa oleviin työnsäntöihin. Periaatteet ja alakohtainen tieto ovat hyödyllisiä ainoastaan, jos ne tavoittavat kehittäjät juuri sillä hetkellä, kun päätöksiä tehdään: toteutuksen, katselmoinnin ja käyttöönnoton aikana – ei jälkikäteen tehtävän auditoinnin muodossa. Tässä osiossa kuvataan, miten energiatietoisuus voidaan sisällyttää tyypillisen kehitystyönkulun eri vaiheisiin.

Työnkulkuun integroitua energiaprofilointia koskeva tutkimus vahvistaa, että työkalut ja prosessit, jotka yhdistävät energiamittarit versionhallintaan, jatkuvaan integraatioon ja visuaalisiin koontinäyttöihin, ovat sekä teknisesti toteutettavissa että kehittäjien hyväksymiä – edellyttäen, että ne vaativat mahdollisimman vähän käyttöönottoimia ja toimivat kehittäjien jo käyttämissä välineissä (Joof et al., 2025).

6.1 Paikallinen kehitys

Edullisin kohta energiatehottomuuden tunnistamiselle on paikallinen kehitys – ennen kuin koodi katselmoidaan, yhdistetään tai otetaan käyttöön. Muutoksen profilointi eristettynä, eli uuden toteutuksen energia- tai resurssijäljen vertaaminen edelliseen version edustavilla testisyöteillä, mahdollistaa nopean kokeilun vähäisellä lisäkuormalla.

Kevyet profilointimenetelmät, jotka käärivät olemassa olevia työkaluja kuten PowerJoular kontainerisoiduiksi, toistettaviksi suoritusympäristöiksi, ovat osoittaneet, että pyyntökohtaiset energiamittarit (CPU-käyttö, muisti, virrankulutus, vasteaika) voidaan kerätä helposti saatavilla olevalla laitteistolla kuten Raspberry Pi ilman erikoisinstrumentointia tai etuoikeutettua pilvipalvelupääsyä (Joof, 2025). Tämä tekee merkityksellisestä energiapalautteesta käytännöllisen yksittäisen kehittäjän tasolla – ei pelkästään erikoistuneissa suorituskykytekniikan tiimeissä.

Keskeinen kurinalaisuuden muoto on lähtötason määrittäminen ennen optimointia. Lähtötason mittaus edustavalla kuormituksella tarjoaa vertailukohtan, johon kaikkia muutoksia voidaan verrata. Ilman sitä parannuksia ei voida varmistaa eikä regressioita havaita.

6.2 Koodikatselmoinnit

Koodikatselmointi on luonteva piste energiatietoisille kysymyksille, jotka edistävät yhteistä vastuuta lisäämättä jäykkiä porttikohia. Katselmoijien ei tarvitse olla

energiamittauksen asiantuntijoita; heillä on oltava yhteinen sanasto ja joukko kehoitteita, jotka tuovat esiin mahdolliset tehottomuudet ennen niiden yhdistämistä.

Hyödyllisiä kysymyksiä katselmointikulttuuriin sisällytettäväksi: - Tuovatko nämä muutokset uusia laskutoimituksia jokaiselle pyynnölle, vai välttävätkö ne työn, jota aiemmin tehtiin tarpeettomasti? - Onko tietojen käyttömalli muuttunut tavalla, joka saattaa kasvattaa kyselymäärää tai hyötykuorman kokoa? - Jos välimuisti lisätään, onko sen koko rajattu ja onko osumataajuuden odotettu perustelevan muistikustannukset? - Vaikuttaako tämä muutos taustaprosessien suoritusiheyteen tai resurssien hallinta-aikaan?

Nämä kysymykset ovat hyödyllisiä ilman profilointidataakin; ne rakentavat tiimin energiantuion ajan myötä ja tuovat esiin ongelmia, joita mittaukset voivat sitten vahvistaa tai poissulkea.

6.3 Versiota seuraava seuranta

Energiajäljän tietojen yhdistäminen tiettyihin koodiversioon (commitit, haarat tai julkaisut) mahdollistaa trendianalyysin ja regressiotunnistuksen, joita pisteytetty profilointi ei pysty tarjoamaan. Yksittäinen mittaus kertoo nykyisen tilan; versiotietoinen seuranta kertoo, parantuuko, heikkeneekö vai pysyykö järjestelmä vakaana kehityshistorian aikana.

Prototyypin järjestelmät, jotka merkitsevät energiamittaukset Git-commit-tunnisteilla ja tallentavat ne kyselyitä tukevaan tietokantaan, ovat osoittaneet, että commit-tason energiavertailu on käytännöllistä ja kehittäjäystävällistä. Kehittäjät voivat tarkkailla tietyn optimoinnin (esim. välimuistin käyttöönoton) energiavaikutusta, vahvistaa sen pysyvyyden peräkkäisissä committeissa ja tunnistaa commitin, jossa regressio otettiin käyttöön – samalla tavalla kuin he nyt seuraavat testivirheitä tai käännösaikoja (Joof et al., 2025; Joof, 2025).

Tämä lähestymistapa kohtelee energiatehokkuutta mitattavana, versioituna ohjelmiston laadullisena ominaisuutena sen sijaan, että se arvioitaisiin epävirallisesti tai jätettäisiin kokonaan arvioimatta.

6.4 CI/CD-integraatio

Jatkuvan integraation ja käyttöönoton putkilinjat ovat skaalautuvuuden kannalta paras piste energiapalautteelle, koska ne suoritetaan automaattisesti jokaisen muutoksen yhteydessä ilman kehittäjän toimia. Energiaprofiloinnin integrointi CI-putkilinjoihin – standardoidun kuormituksen suorittaminen ja energiamittareiden kerääminen jokaisen push- tai pull request -tapahtuman yhteydessä – laajentaa versiotietoinen seurannan käsitteen automatisoidun rakentamisprosessin osaksi.

Webhook-käynnistetty profilointi, jossa Git push -tapahtuma automaattisesti ottaa testattavan API:n käyttöön kontainerisoidussa ympäristössä ja suorittaa ennalta määritellyn kuormituksen, on validoitu käytännölliseksi arkkitehtuuriksi tähän tarkoitukseen. Prototyypin arvioinnissa koko putkilinja commitista koontinäytön päivitykseen valmistui alle 30 minuutissa, mukaan lukien konttien rakentaminen, kuormituksen suoritus ja datan koostaminen (Joof et al., 2025).

Automaattinen energiapalaute CI:ssä tulisi suunnitella huolellisesti: - **Tiedottaminen ensin:** Esitä energiamittarit raporttina testitulosten rinnalla ennen automaattisten virheiden käyttöönottoa energiaregressioille. Tiimit tarvitsevat aikaa kehittääkseen intuitiota siitä, mikä muutos on merkityksellinen, ennen kuin energiaportit ovat hyödyllisiä. - **Valikoiva laajuus:** Profiloi pääteipisteet tai komponentit, joihin muutos todennäköisimmin vaikuttaa, sen sijaan että ajettaisiin täysi kuormitustesti jokaisella commitilla. - **Kontekstittietoiset kynnyksarvot:** Regressiot tulisi merkitä suhteessa liukuvaan lähtötasoon, ei kiinteään absoluuttiseen rajaan, jotta huomioidaan kuormituksen ja laitteiston tilan vaihtelu.

6.5 Visualisointi ja viestintä

Raakaenergiamittaukset (teholukemat, CPU-prosenttiosuudet, joulet per pyyntö) eivät ole useimpien kehittäjien suoraan tulkittavissa. Tehokas visualisointi kääntää nämä luvut päätöksiksi tukevaan muotoon: trendiviivat, jotka osoittavat parantuuko energiajälki commitien välillä, palkkikaaviot, jotka vertailevat suunnitteluvaihtoehtoja rinnakkain, ja värikoodatut indikaattorit (esim. punainen/keltainen/vihreä), jotka viestivät regressiosta tai parannuksesta silmäyksellä.

Kehittäjien palaute prototyypin arvioinneista vahvistaa johdonmukaisesti, että energiadatan visuaaliset esitykset lisäävät merkittävästi ymmärrystä ja halukkuutta toimia tiedon perusteella. Koontinäkyvä, joka osoittaa, kuinka välimuistin käyttöönotto vähensi sekä CPU-käyttöä että virrankulutusta peräkkäisten commitien välillä, ”teki energiadatasta ymmärrettävää” tavalla, jota raakalokituloste ei mahdollistanut (Joof et al., 2025). Visualisoinnit tukevat myös viestintää sidosryhmien kanssa, jotka eivät ole suoraan osallisina toteutuksessa, tehden

teknisten päätösten energiavaikutukset luettaviksi tuoteomistajille, arkkitehteille ja kestävyysasuuntuneelle johdolle.

Tehokkaat energiakoontinäytöt sisältävät: päätepesteykohtaiset resurssierittelyn, commit-kohtaiset trendikuvaajat, haaravertailunäkymät suunnitteluvaihtoehtojen arviointiin sekä testiajoa kohden kulutetun kokonaisenergian pitkäaikaisempaa raportointia varten.

6.6 Organisatorinen käyttöönotto

Tekniset työkalut ovat välttämättömiä mutta eivät riittäviä. Vihreä koodaus muuttuu kestäväksi käytännöksi, kun sillä on kulttuurinen ja organisatorinen tuki: kun tiiminvetäjät kohtelevat energiaa ensisijaisena laadullisena huolenaiheena, kun kehittäjillä on pääsy koulutukseen siitä, mitä mittarit tarkoittavat ja miten niihin tulee reagoida, sekä kun kestävyystavoitteet heijastuvat siihen, miten projekteja rajataan ja arvioidaan.

Tutkimus vahvistaa, että kehittäjät, joilla on henkilökohtainen motivaatio vähentää energiankulutusta, eivät usein pysty toimimaan tehokkaasti ilman institutionaalista tukea: työkaluja, mittareita, johdon tukea ja määriteltyjä tavoitteita (Joof, 2025). Organisaatioiden, jotka haluavat juurruttaa vihreän koodauksen käytäntöjä, tulisi siksi investoida kaikkiin kolmeen: työkaluihin, jotka tekevät energiasta näkyvän, koulutukseen, joka rakentaa intuitiota, sekä organisatoriseen yhdenmukaistamiseen, joka kohtelee energiatehokkuutta yhteisenä insinööriyövastuuna eikä yksilöllisenä huolenaiheena.

Viitteet

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyedeji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

7. Kompromissit, rajoitukset ja suunnittelujännitteet

Vihreä koodaus edellyttää energiäjäljen tasapainottamista suhteessa muihin ohjelmiston laatuominaisuuksiin. Tässä kuvatut jännitteet ovat todellisia, eikä niitä voida ratkaista pelkillä nyrkkisäännöillä. Jokainen niistä vaatii empiiristä arviointia kussakin asiayhteydessä.

7.1 CPU vs. muisti

CPU-työn vähentäminen lisää usein muistinkäyttöä. Tulosten ennakkolaskenta ja välimuistiin tallentaminen esimerkiksi välttää toistuvan laskennan heap-tilan kustannuksella. Nettovaikutus energiankulutukseen riippuu käyttömallista: muistia on suhteellisen edullista *pitää varattuna*, mutta kallista *allokoida, kopioida ja kerätä roskana* suurella tahdilla.

Päätösheuristiikka: Suosi muisti-intensiivistä vaihtoehtoa, kun dataan viitataan toistuvasti rajatun aikajakson sisällä ja välimuistin osumataajuus on osoitetusti korkea. Suosi laskenta-intensiivistä vaihtoehtoa, kun viittaukset ovat harvoja, datan volyyymi tekee välimuistiin tallentamisesta epätaloudellista tai välimuistissa olevien objektien aiheuttama GC-paine ylittää laskennan säästöt. Mittaa molemmat lähestymistavat edustavan kuorman alla ennen päätöksentekoa.

7.2 Verkko vs. laskenta

Tiedonsiirron vähentäminen voi vaatia lisälaskentaa — esimerkiksi hyötykuorman pakkaamista ennen lähetystä tai datan esikoostamista palvelinpuolella raakatietueiden siirtämisen sijaan. Toisaalta laskennan ohittaminen datan nopeampaa lähettämistä varten pitää verkon aktiivisena kauemmin ja voi lisätä laitteen puoleisia käsittelykustannuksia.

Päätösheuristiikka: Kun käyttäjän kokemaa viive ei ole rajoittava tekijä, suosi siirrettävän datamäärän vähentämistä. Kun viive on kriittinen, vertaile, palautuuko lisäkäsittelyn (pakkaus, koostaminen) energiakustannus lyhentyneen lähetysajan

kautta. Huomaa, että verkon energiakustannukset vaihtelevat merkittävästi langallisen, Wi-Fi- ja mobiiliradioympäristöjen välillä. Tämä kompromissi on havaittu empiirisesti API-tason profiloinnissa: GZIP-pakkauksen lisääminen API-vastauksiin pienensi verkon hyötykuormaa mutta kasvatti CPU-kuormaa verrattuna välimuistitarkaisuun, mikä johti korkeampaan tehonkulutukseen (250 mW vs. 240 mW) siirtokoon pienenemisestä huolimatta — tämä osoittaa, ettei pakkaaminen ole ehdottomasti hyödyllistä (Joof et al., 2025; Joof, 2025).

7.3 Tehokkuus vs. ylläpidettävyys

Voimakkaasti optimoitu koodi on usein vaikeampi ymmärtää, testata ja muuttaa. Energiaoptimointi, joka kytkee komponentit tiukasti yhteen, poistaa hyödyllisiä abstraktioita tai nojaa dokumentoimattomaan laitteistokäyttäytymiseen, voi pienentää energiajälkeä nyt mutta kasvattaa sitä myöhemmin vikojen ja uudelleentyöstämisen kautta.

Päätösheuristiikka: Sovella ensin energiaperusteisia optimointeja, jotka eivät merkittävästi lisää koodin monimutkaisuutta. Kun invasiivisempi optimointi on välttämätön, dokumentoi säilytettävä suorituskykyominaisuus, eristä optimoitu koodi selkeärajaiseen rajapintaan ja lisää vertailutestit, jotka paljastavat regressiot, jos optimointi poistetaan tai ohitetaan myöhemmin.

7.4 Paikalliset vs. järjestelmänlaajuiset vaikutukset

Yksittäisen komponentin optimointi eristyksissä voi siirtää energiankulutusta muualle järjestelmässä sen sijaan, että se poistaisi kulutuksen kokonaan. Nopeampi asiakaspuolen operaatio voi tuottaa enemmän palvelinpyyntöjä. Tehokkaampi tietokantakysely voi lisätä sovelluskerroksen käsittelyä. Pienennetty verkon hyötykuorma voi vaatia enemmän CPU:ta vastaanottavalla päällä.

Päätösheuristiikka: Jäljitä ennen komponentin optimointia tietovuo järjestelmän läpi alaspäin suuntautuvien vaikutusten tunnistamiseksi. Priorisoi optimointeja, jotka vähentävät koko järjestelmän energiankulutusta, eivät vain paikallisia mittareita. Kun järjestelmänlaajuinen mittaus ei ole käytännöllistä, varmista vähintään, etteivät ylä- ja alajuoksun komponentit kuormitu haitallisesti muutoksen seurauksena.

7.5 Mittausepävarmuus

Energianmittaukseen liittyy luontaisia rajoituksia. Ohjelmistosta luettavat laskurit (kuten Intel RAPL) mittaavat prosessori- ja DRAM-alijärjestelmiä, mutta jättävät ulkopuolelle tallennusmedian, verkon ja oheislaitteet. Pilvi- ja virtualisointiympäristöt eivät usein tarjoa lainkaan pääsyä laitteiston energialaskureihin. Tulokset vaihtelevat CPU-taajuudensäädön, lämpörajoittamisen, taustakuorman ja mittaustarkkuuden mukaan.

Päätösheuristiikka: Raportoi suhteelliset parannukset lähtötason ja ehdokkaan välillä identtisissä olosuhteissa absoluuttisten energiaarvojen sijaan. Toista mittaukset useilla ajoilla ja mahdollisuuksien mukaan eri laitteistokokoonpanoilla ennen johtopäätösten tekemistä. Ilmoita mittausolosuhteet eksplisiittisesti: käytetty työkalu, järjestelmän tila, kuorma ja toistojen lukumäärä. Pidä tulosta alustavana, kunnes se on validoitu edustavan tuotantoympäristön olosuhteissa.

7.6 Vähenevät tuotot

Kun merkittävimmät tehottomuudet — kuten turha laskenta, tarpeeton tiedonsiirto ja resurssien joutokäynti — on poistettu, lisäoptimointi tuottaa asteittain pienempiä energiasäästöjä kasvavilla toteutuskustannuksilla. Ensimmäinen 20 % optimointipanostuksesta kattaa usein 80 % saavutettavissa olevasta parannuksesta.

Päätösheuristiikka: Priorisoi toimenpiteet arvioidun energiavaikutuksen mukaan suhteessa toteutusvaivaan. Käytä profilointia energiajäljen pääasiallisten tekijöiden tunnistamiseen ennen optimointiin panostamista. Lopeta komponentin optimointi, kun lisäparannukset edellyttävät kohtuutonta monimutkaisuutta tai riskiä, ja suuntaa panostus mittauksen perusteella tunnistettuihin suuremman vaikutuksen alueisiin.

Lähteet

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyedeji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

8. Ohjelmiston energialajinjaljki

Ohjelmiston energialajinjaljin ymmärtäminen edellyttää siirtymistä pois intuitiivisista käsityksistä siitä, mikä koodi on "nopeaa" tai "tehokasta", kohti systemaattista ymmärrystä siitä, miten ohjelmiston toiminta muuntuu energiankulutukseksi. Tämä luku tarjoaa käsitteellisen perustan tuolle ymmärrykselle.

8.1 Mitä ohjelmiston energialajinjaljki tarkoittaa?

Ohjelmiston energialajinjaljki on se kokonaisenergiämäärä, jonka laitteistojärjestelmä kuluttaa suoraan seurauksena tietyn ohjelmiston suorittamisesta määritellyllä ajanjaksolla tai työkuormalla. Se ei ole lepotilassa olevan koodin ominaisuus; se ilmenee suorituksen aikana ja riippuu siitä, mitä ohjelmisto tekee, kuinka usein, millä laitteistolla ja millaisissa olosuhteissa.

Tämä suoritusaikainen luonne erottaa ohjelmiston energialajinjaljin elinkaarilähtöisistä näkökulmista, jotka ottavat huomioon myös laitteistovalmistukseen, konesalirakentamiseen tai käytöstäpoistoon sitoutuneen energian. Nämä näkökohdat ovat todellisia ja merkittäviä, mutta ne ovat suurelta osin ohjelmistokehittäjän toteutuspäätösten ulottumattomissa. Suoritusaikainen lajinjaljki sen sijaan muotoutuu suoraan ohjelmiston suunnitteluvaiheista ja on siksi vihreän koodauksen käytännön ensisijainen kohde.

Jalajinjaljki on myös luonteeltaan **työkuormariippuvainen**. Sama koodipohja voi tuottaa huomattavasti erilaisen energialajinjaljin syötteen koon, samanaikaisten käyttäjien määrän, pyyntöjen tiheyden ja välimuistissa olevan datan määrän mukaan. Tämä tarkoittaa, että energiaa ei voida mielekkäästi luonnehtia pelkästään koodia tarkastelemalla; se edellyttää mittaamista edustavissa olosuhteissa.

8.2 Miten ohjelmisto kuluttaa energiaa?

Ohjelmisto ei kuluta energiaa suoraan. Se saa laitteistokomponentit tekemään työtä, ja nuo komponentit kuluttavat energiaa. On olennaista ymmärtää, mitkä komponentit ovat osallisina ja mikä ajaa niiden kulutusta, jotta voidaan tunnistaa, missä optimointityöllä on suurin vaikutus.

Proessori (CPU/GPU/TPU) Proessori on tyypillisesti hallitseva energiankuluttaja aktiivisen laskennan aikana. Dynaaminen teho, eli transistorien kytkennästä aiheutuva energiankulutus, skaalautuu kellotaajuuden ja aktiivisten piirien määrän mukaan. Modernit prosessorit käyttävät taajuus- ja jänniteskaalausta tehon vähentämiseksi alhaisella kuormituksella, mutta korkea käyttöaste ajaa ne kohti huipputehon rajaa. Ohjelmisto, joka pitää prosessorit korkean käyttöasteen tilassa pitkiä aikoja, vaikuttaa suhteettomasti energiankulutukseen.

Muisti Dynaaminen RAM kuluttaa energiaa sekä käytön että valmiustilan aikana. Muistin luku- ja kirjoitusoperaatiot, erityisesti ne, jotka ohittavat CPU-välimuistin ja ulottuvat päämuistiin, ovat operaatiokohtaisesti huomattavasti energiantensivisempiä kuin L1- tai L2-välimuistissa pysyvät operaatiot. Ohjelmisto, jolla on heikko datan paikallisuus, suuri työskentelymäärä tai korkea allokointitahti ja roskienkeruu, lisää DRAM-energiankulutusta.

Verkkoiliitännät Datan lähettäminen ja vastaanottaminen verkkoliitännän kautta vaatii energiaa suhteessa siirretyn datan määrään, lisätyn kiinteällä ylikuormituksella yhteyden muodostamisesta ja protokollaneuvottelusta. Mobiiliympäristöissä radioliitännöillä (LTE, 5G, Wi-Fi) on erityisen korkeat energiakustannukset suhteessa siirrettyyn dataan, ja ne pysyvät kohonneessa tehotilassa jonkin aikaa lähetyksen päättymisen jälkeen — ilmiö tunnetaan nimellä *radiokaista* — mikä tekee monista pienistä lähetyksistä huomattavasti kalliimpia kuin harvemmista suuremmista (Pathak et al., 2012).

Tallennus Puolijohdetallennus kuluttaa energiaa luku- ja kirjoitusoperaatioiden aikana, ja kirjoitukset ovat tyypillisesti kalliimpia kuin luvut. Pyörivällä levyllä on lisäksi mekaanisia energiakustannuksia. Ohjelmisto, joka suorittaa tarpeetonta I/O:tä — kuten kirjoittaa tarpeettomia lokeja, lukee uudelleen muuttumattomia tiedostoja tai suorittaa usein pieniä satunnaisia kirjoituksia — lisää tallennuksen energiankulutusta suhteessa käyttötiheyteen ja -volyymiin.

Näyttö Käyttäjälle näkyvissä sovelluksissa näyttö voi olla merkittävä energiankuluttaja, erityisesti mobiililaitteissa, joissa näytön kirkkaus ja renderöinnin monimutkaisuus vaikuttavat suoraan akun kulumiseen. Ohjelmisto, joka pitää

näytön tarpeettomasti aktiivisena, renderöi täydessä resoluutiossa kun alempi riittäisi, tai käynnistää tiheästi näyttöpäivityksiä näkymättömiin muutoksiin, lisää näytön energiankulutusta.

8.3 Energian kohdentaminen ohjelmistolle

Keskeinen haaste vihreässä koodauksessa on se, että energia mitataan järjestelmätasolla kokonaisvirtalähteen tehona, kun taas vastuu kulutuksesta jakautuu käyttöjärjestelmän, ajonaikaisympäristön, kehysten, sovellusloodin ja samanaikaisesti suoritettavien prosessien kesken. Tietyn osuuden kohdentaminen järjestelmäenergiasta tietylle ohjelmistokomponentille on teknisesti vaikeaa ja sisältää merkittävää epävarmuutta.

Käytännössä käytetään useita lähestymistapoja:

Laitteiston suorituskykylaskurit Modernit prosessorit paljastavat energianmittausrekisterit, joita ohjelmisto voi lukea. Intelin Running Average Power Limit (RAPL) -rajapinta tarjoaa esimerkiksi pakettikohtaisia ja toimialuekohtaisia (CPU, DRAM, uncore) energialukemia suurella tarkkuudella. Nämä ovat suurimmat ohjelmistosta käytettävissä olevat energiamittaukset kaupallisella laitteistolla, mutta ne kattavat vain prosessori- ja muistialijärjestelmät eivätkä ole saatavilla kaikissa virtualisoiduissa tai pilviympäristöissä.

Järjestelmätason tehomittaus Ulkoiset tehomittarit mittaavat järjestelmän kokonaisenergiaa sähköpistokkeesta tai laitekotelon tasolta. Tämä kattaa kaikki komponentit, mutta ei kohdistu yksittäisiin prosesseihin tai ohjelmistokomponentteihin. Se soveltuu parhaiten kahden identtissä olosuhteissa eristyksessä suoritettavan työkuormaversion vertailuun.

Mallipohjainen arviointi Kun suora mittaus ei ole saatavilla — kuten pilvi- ja jaetussa infrastruktuurissa on yleistä — energiankulutus voidaan arvioida CPU-käyttöasteesta, muistin käytöstä ja työkuorman ominaisuuksista empiirisillä malleilla. Tällaisten arvioiden tarkkuus vaihtelee huomattavasti ja riippuu taustalla olevan laitteistokarakterisaation laadusta. Cloud Carbon Footprint -projekti ja Green Software Foundationin Software Carbon Intensity (SCI) -määrittely ovat esimerkkejä kehyksistä, jotka formalisoivat tämän lähestymistavan.

Mittauslähestymistavasta riippumatta kohdentaminen on luotettavinta, kun yksittäinen työkuorma suoritetaan eristettynä tunnetulla laitteistolla. Tuotantoympäristöissä kohdentaminen on aina osittaista, ja tuloksia tulee tulkita sen mukaisesti (ks. myös [Luku 7, Mittausepävarmuus](#)). Käytännöllinen demonstraatio API-tason kohdentamisesta yhdistää kontaineroidun suorituksen — joka eristää kohdetyökuorman taustapalveluista — ohjelmiston energiaprofiloijaan kuten PowerJoular, jolla voidaan kohdentaa tehonkulutus tiettyihin päätepisteisiin ja commit-versioihin. Tämä lähestymistapa saavuttaa hyvän yhteneväisyyden suorien profiloijalukemien kanssa (Pearsonin $r = 0,94$) vaatien vain kaupallista laitteistoa ja avoimen lähdekoodin työkaluja (Joof et al., 2025; Joof, 2025).

8.4 Energia, teho ja hiili: käsitteiden selventäminen

Nämä kolme toisiinsa liittyvää käsitettä sekoitetaan usein toisiinsa, mutta ne ovat erillisiä ja jokaisella on eri merkitys mittaamisen ja optimoinnin kannalta.

Teho (mitataan watteina, W) on energiankulutuksen nopeus hetkellä. Prosessi, joka kuluttaa 50 W, kuluttaa energiaa sillä nopeudella. Teho yksin ei kerro kokonaiskulutettua energiaa, sillä se riippuu siitä, kuinka kauan tehoa otetaan.

Energia (mitataan jouleina, J, tai kilowattitunteina, kWh) on tehon ja ajan tulo: $E = P \times t$. Prosessi, joka kuluttaa 50 W 10 sekunnin ajan, kuluttaa 500 J. Energia on oikea yksikkö työkuorman suorituksen kokonaiskustannuksen vertailuun, koska se ottaa huomioon sekä tehonkulutuksen että keston. Nopeampi mutta tehoahnaampi toteutus ei välttämättä ole energiatehokkaampi kuin hitaampi, pienempikulutteinen vaihtoehto, koska kokonaisenergia — tehon integraali ajan yli — määrää jalanjäljen.

Hiilidioksidipäästöt (mitataan grammoina tai kilogrammoina CO₂-ekvivalenttia, CO₂e) kuvaavat energiankulutuksen ilmastovaikutusta. Sama kilowattitunti sähköä tuottaa erilaisia hiilidioksidipäästöjä lähteestä riippuen: lähes nollaan aurinko- tai tuulivoimalla, ja huomattavasti enemmän hiilellä tai kaasulla. Hiilivoimakkuus, mitattuna grammoina CO₂e per kilowattitunti, vaihtelee sähköverkkoalueen, vuorokauden ajan ja vuodenajan mukaan. Energiajalanjäljen ja hiilijalanjäljen optimointi osoittavat yleensä samaan suuntaan, mutta suhde ei ole kiinteä, ja työkuormien maantieteellinen tai ajallinen siirto voi vähentää hiilipäästöjä energiansäästöstä riippumatta.

Useimmissa ohjelmiston optimointitöissä **energia** on suoraan toimenpidekelpoinen mittari. Hiilitilitys on relevanteinta infrastruktuuri- ja käyttöönottostrategian tasolla, jossa pilvipalvelualueen, energiahankinnan ja työkuorman ajoituksen valinnat ovat vuorovaikutuksessa sähköverkon hiilivoimakkuuden kanssa.

8.5 Ohjelmiston energiajalanjälkeä muovaavat tekijät

Ohjelmiston energiajalanjälkeä ei määrää koodi yksin. Se on ohjelmiston toiminnan ja suoritusympäristön välisen vuorovaikutuksen tuote. Keskeisimmät tekijät ovat:

Algoritminen monimutkaisuus Algoritmin asyymptootinen monimutkaisuus määrää, miten sen resurssienkulutus skaalautuu syötteen koon myötä. $O(n^2)$ -algoritmi sovellettuuna suureen tietojoukkoon voi kuluttaa kertaluokkia enemmän energiaa kuin $O(n \log n)$ -vaihtoehto. Algoritmivalinnoilla on siten suurin vaikutus energiajalanjälkeen mittakaavassa, ja ne tehdään kehityksen alkuvaiheessa, mikä tekee niistä energianäkökulmasta tärkeimpiä arvioitavia päätöksiä (Pereira et al., 2017). Tämä vaikutus on mitattavissa jopa pienessä mittakaavassa: mukautetun sisäkkäissilmukkaisten lajittelun korvaaminen kielen sisäänrakennetulla lajittelulla backend-API:ssa vähensi tehonkulutusta 23 % ja CPU-käyttöastetta 34,5 %:sta 26,1 %:iin standardoidulla pyyntötyökuormalla (Joof et al., 2025; Joof, 2025).

Datan käyttökuviot Sillä, toimivatko laskelmat CPU-välimuistissa, päämuistissa vai levyllä olevalla datalla, on suuri vaikutus energiankulutukseen. Välimuistiystävälliset tietorakenteet ja käyttökuviot, erityisesti ne, jotka osoittavat spatiaalista ja ajallista paikallisuutta, ovat huomattavasti energiatehokkaampia kuin ne, jotka hajottavat muistikäyttöä suurille osoitealueille.

Rinnakkaisuus ja rinnakkaislaskenta Rinnakkaislaskenta lisää laitteiston käyttöastetta ja voi lyhentää reaaliaikaista suoritusaikaa, mutta se ei välttämättä vähennä energiankulutusta. Jos useat säikeet suorittavat redundanttia työtä tai jos synkronointiylimäärä on suuri, rinnakkais suoritus voi kuluttaa enemmän energiaa kuin saman tehtävän peräkkäinen suoritus. Toisaalta hyvin jäsenneltä rinnakkaislaskenta voi parantaa energiatehokkuutta suorittamalla työn nopeammin ja mahdollistamalla laitteiston palautumisen alhaisempiin tehotilaan aiemmin.

Laitteiston ja ajonaikaisen ympäristön ominaisuudet Sama koodi tuottaa erilaisia energiajalanjälkiä eri laitteistoilla. Prosessorin mikroarkkitehtuuri, muistihierarkian rakenne ja laitteistoalustan energiaproportionaalisuus (kuinka läheisesti valmiusteho skaalautuu kuorman mukaan) vaikuttavat kaikki mitattuun jalanjälkeen. Ajonaikaiset ominaisuudet kuten JIT-käännös, roskienkeruu ja tulkin ylikuormitus lisäävät lisää vaihtelua.

Käyttöönotto ja skaalaus Käynnissä olevien instanssien määrä, niiden palvelema kuorma ja pyyntöjen välinen valmiusaika vaikuttavat kaikki kokonaisenergiankulutukseen. Erittäin optimoitu yksittäinen instanssi, joka toimii hyvin alhaisella käyttöasteella, voi kuluttaa enemmän energiaa kuin kohtuullisesti optimoitu instanssi, joka toimii korkealla käyttöasteella, palvelimen käynnissäpitämisen kiinteän energiakustannuksen vuoksi kuormasta riippumatta.

Viitteet

Green Software Foundation (2023) *Software Carbon Intensity (SCI) Specification*. Version 1.0. Available at: <https://sci.greensoftware.foundation>

Pathak, A., Hu, Y.C. and Zhang, M. (2012) *Where is the energy spent inside my app? Fine-grained energy accounting on smartphones with Eprof*. Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys), pp.29–42.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J.P. and Saraiva, J. (2017) *Energy efficiency across programming languages: How do energy, time, and memory relate?* Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering, pp.256–267.

Treiber, M. (2015) *RAPL in action: Experiences in using RAPL for power measurements*. ACM Transactions on Modeling and Performance Evaluation of Computing Systems, 1(1), pp.1–26.

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.

9. Ohjelmiston kestävyyskädenjälki

Vihreä koodaus, kuten tässä oppaassa kuvataan, keskittyy ensisijaisesti ohjelmiston suorittamisen suorien energiakustannusten vähentämiseen. Ohjelmistolla on

kuitenkin myös välillinen ympäristöulottuvuus: järjestelmät, joita se mahdollistaa, käyttäytymiset, joita se välittää, ja fyysiset prosessit, jotka se korvaa tai parantaa, voivat tuottaa myönteisiä ympäristövaikutuksia, jotka ylittävät huomattavasti ohjelmiston oman energiakustannuksen. Tähän myönteiseen ympäristöpanokseen viitataan **ohjelmiston kestävyyskädenjälkeen**.

9.1 Jalanjäljestä kädenjälkeen

Kädenjäljen käsite on peräisin kestävyystutkimuksesta vastakohdaksi tutummalle *jalanjäljelle*. Siinä missä jalanjälki mittaa toiminnan ympäristöarastusta — kulutettuja resursseja ja syntyviä päästöjä — kädenjälki mittaa sen luomaa ympäristöhyötyä: jalanjälkeä, joka vältetään, vähennetään tai palautetaan toiminnan seurauksena (Biggs et al., 2020).

Ohjelmistoon sovellettuna jako hahmottuu selkeästi kahdeksi erilliseksi kysymykseksi:

- **Jalanjälkikysymys:** Kuinka paljon energiaa tämä ohjelmisto kuluttaa toimiessaan?
- **Kädenjälkikysymys:** Minkä ympäristöhaitan tämä ohjelmisto estää tai vähentää toimiessaan tarkoitetulla tavalla?

Nämä ovat toisistaan riippumattomia. Ohjelmistolla, jolla on suuri jalanjälki, voi olla suuri myönteinen kädenjälki — esimerkiksi laskennallisesti intensiivinen reittioptimaatiojärjestelmä, joka säästää miljoonia litroja polttoainetta vuosittain. Ohjelmistolla, jolla on pieni jalanjälki, voi olla merkityksetön kädenjälki — esimerkiksi hyvin optimoitu mutta toiminnallisesti triviaali sovellus. Ja ohjelmistolla voi joissain tapauksissa olla negatiivinen kädenjälki, kun se mahdollistaa toimintoja tai kulutuskuvioita, jotka lisäävät kokonaisympäristöhaittaa.

Vihreän koodauksen käytäntö käsittelee jalanjälkiulottuvuutta. Kädenjälkiulottuvuuden ymmärtäminen vaatii erilaisen ja laajemman kysymyksen esittämistä: *mitä tämä ohjelmisto mahdollistaa, ja mikä on sen ympäristöseuraus?*

9.2 Miten ohjelmisto luo myönteistä ympäristövaikutusta?

Ohjelmisto luo ympäristöhyötyä useiden eri mekanismien kautta. Seuraavat kategoriat havainnollistavat vaikutusten kirjoa fyysisten prosessien korvaamisesta fyysisten järjestelmien optimointiin.

Dematerialisaatio Ohjelmisto voi korvata fyysisiä tavaroita ja palveluita, joiden tuottaminen ja toimittaminen vaatii energiaa, materiaaleja ja logistiikkaa. Digitaaliset asiakirjat korvaavat tulostetut. Videoneuvottelut korvaavat lennot. Suoratoistomedia korvaa fyysiset levyt ja niiden tuottamiseen ja jakeluun tarvittavat toimitusketjut. Dematerialisaatiosta saatava ympäristöhyöty on todellinen, mutta ei automaattinen: se riippuu siitä, otetaanko digitaalinen vaihtoehto käyttöön fyysisen sijaan eikä sen lisäksi (Hilty and Aebischer, 2015).

Etätyön ja hajautetun yhteistyön mahdollistaminen Etätyötä tukevat ohjelmistoalustat vähentävät päivittäisen pendelöinnin ja työmatkustamisen tarvetta, jotka molemmat ovat merkittäviä liikenteen päästöjen lähteitä. Ympäristöhyöty riippuu siitä, tapahtuuko matkakorvaus todellisuudessa, ja siitä tasapainottaa jonkin verran lisääntynyt asumisenergiankäyttö ja yhteistyöohjelmiston oma energiakustannus. Nettovaikutus on yleensä myönteinen, kun pitkät työmatkasukkuloinnit tai lennot korvataan (Hook et al., 2020).

Fyysisten järjestelmien optimointi Jotkut suurimmista kädenjälkimahdollisuuksista syntyvät ohjelmistosta, joka tekee fyysisestä infrastruktuurista tehokkaamman. Esimerkkejä ovat: - *Älykkäät rakennusten hallintajärjestelmät*, jotka säätelevät dynaamisesti lämmitystä, jäähdytystä ja valaistusta käytön ja sään perusteella, vähentäen rakennusten energiankulutusta, joka kattaa huomattavan osan maailman energiankäytöstä. - *Älykkäät liikennejärjestelmät*, jotka optimoivat liikennevirtoja, vähentävät tyhjääkäyntiä ja parantavat joukkoliikenteen aikataulutusta, alentaen polttoaineen kulutusta ajoneuvokannassa. - *Täsmämaatalousplatformit*, jotka käyttävät anturidataa ja mallinnusta levittämään vettä, lannoitteita ja torjunta-aineita vain sinne ja silloin kuin niitä tarvitaan, vähentäen resurssien kulutusta ja siihen liittyviä päästöjä. - *Sähköverkon hallinta- ja kysyntäjoustoohjelmit*, jotka integroivat uusiutuvia energianlähteitä, siirtävät joustavaa kysyntää uusiutuvan energian tuotannon huippu-aikoihin ja vähentävät tuuli- ja aurinkovoiman leikkausta.

Tieteellisen ja teknisen edistymisen mahdollistaminen Ohjelmistotyökalut, jotka nopeuttavat puhtaan energian, materiaalitieteen, ilmastomallituksen ja vähähiilisen insinöörityön tutkimusta, voivat tuottaa kädenjälkivaikutuksia, jotka kertautuvat ajan myötä. Simulaatiotyökalun ympäristöarvoa, joka alentaa tehokkaamman akkukemian kehittämiskustannuksia, ei voida tavoittaa ohjelmiston omassa energiajalanjäljessä; se ilmenee sen mahdollistaman tutkimuksen jatko vaikutusten kautta.

9.3 Kädenjälkiväitteiden arviointi

Kädenjäljen käsite on arvokas mutta helposti väärinkäytetty. Koska kädenjälkivaikutukset ovat välillisiä ja sisältävät kontrafaktuaaleja (mitä *olisi tapahtunut* ilman ohjelmistoa), ne ovat luonteeltaan vaikeampia mitata ja todentaa kuin jalanjälkivaikutukset. Useat periaatteet ohjaavat tiukkaa kädenjälkiarviointia.

Additionaalisuus Aito kädenjälki edellyttää, että ympäristöhyöty ei olisi toteutunut ilman ohjelmistoa. Jos reittiioptimaatiojärjestelmä otetaan käyttöön korvaamaan järjestelmää, joka jo suoritti riittävää optimaatiota, uuden järjestelmän marginaalihyöty on relevantti kädenjälki, ei minkä tahansa reittiioptimaation kokonaisvältettyjä päästöjä. Additionaalisuus on usein liioiteltu kestävyysraportoinnissa.

Rebound-vaikutukset Ohjelmiston mahdollistamat tehokkuusparannukset voivat käynnistää lisääntymistä tehokkuutta tekevässä toiminnassa, osittain tai kokonaan kumoamalla ympäristöhyödyn. Tehokkaampi navigointi vähentää polttoaineen kulutusta matkaa kohden, mutta halvempi, nopeampi tai kätevämpien liikennemahdollisuuksien seurauksena matkojen kokonaismäärä voi kasvaa. Tehokkaampi datanpakkaus vähentää energiakustannusta gigatavua kohden, mutta alhaisempi kustannus gigatavua kohden lisää kokonaistietokoneiden kulutusta. Nämä *rebound-vaikutukset* on otettava huomioon arvioitaessa nettokädenjälkeä.

Kontrafaktuaalinen tarkkuus Kädenjälkiväite edellyttää sen määrittämistä, mikä vaihtoehtoinen skenaario on. "Ohjelmisto X vähentää hiilipäästöjä" ei ole täydellinen väite; se on verrattava määritellyn vertailutasoon. Onko vaihtoehto olla käyttämättä lainkaan ohjelmistoa? Vanhempi versio ohjelmistosta? Manuaalinen prosessi? Kilpailijan tuote? Kädenjäljen suuruus riippuu voimakkaasti siitä, mikä kontrafaktuaali valitaan, ja valinnan on oltava eksplisiittinen ja perusteltu.

Laajuus ja aikahorisontti Kädenjälkivaikutukset voivat toimia hyvin erilaisilla aikaskaalavilla. Reittiioptimaatiojärjestelmän säästämä polttoaine realisoituu välittömästi, kun taas puhtaan energian tutkimuksen nopeuttamisen vaikutus voi materialisoitua vuosikymmenten kuluessa. Pitkät aikahorisontit lisäävät epävarmuutta. Ole eksplisiittinen kädenjälkiväitteen laajuuden suhteen: mitkä vaikutukset sisällytetään, millä ajanjaksolla ja millä epävarmuudella.

9.4 Nettovaikutus ja jalanjäljen ja kädenjäljen suhde

Useimmissa ohjelmistojärjestelmissä suora energijalanjälki on pieni suhteessa toimitettuun toiminnalliseen arvoon. Hyvin suunniteltu älytermostaattijärjestelmä saattaa esimerkiksi kuluttaa muutaman kilowattitunnin vuodessa viestintään ja laskentaan mahdollistaessaan samalla lämmitys- ja jäähdytysäästöjä sadoissa kilowattitunneissa vuodessa hallinnoimissaan rakennuksissa. Tällaisissa tapauksissa kädenjälki dominoi selvästi jalanjälkeä, ja ohjelmiston nettopäästövaikutus on selvästi myönteinen.

Tämä suotuisa suhde ei kuitenkaan ole universaali, eikä sitä pitäisi olettaa. Ohjelmisto, joka kuluttaa merkittävästi energiaa — kuten laajamittainen datankäsittely, jatkuva videosuoratoisto tai tekoälymallien koulutus — voi vaatia positiivista nettovaikutusta vain, jos sen kädenjälkivaikutukset ovat todellisia, additionaalisia ja riittävän suuria jalanjäljen kumoamiseen. Tähän suuntaan esitetyt väitteet tulisi pitää samaan todistushaaraan tasoon kuin mikä tahansa muu empiirinen väite.

Ohjelmistokehittäjille ja arkkitehteille käytännön implikaatio on tämä: **jalanjäljen vähentäminen ja kädenjäljen ymmärtäminen ovat toisiaan täydentäviä, eivät kilpailevia huolenaiheita.** Järjestelmä, jolla on aito myönteinen kädenjälki, voi maksimoida nettohyötynsä minimoimalla myös toiminnallisen jalanjälkensä. Ympäristövaikutusten tiukka, kvantitatiivinen, kontrafaktuaalinen ja rebound-vaikutukset huomioiva arviointikuri on sama kuri riippumatta siitä, sovelletaanko sitä jalanjälkeen vai kädenjälkeen.

Viitteet

Biggs, R., de Vos, A., Preiser, R., Clements, H., Maciejewski, K. and Schlüter, M. (eds.) (2020) *The Routledge Handbook of Research Methods for Social-Ecological Systems*. Routledge.

Hilty, L.M. and Aebischer, B. (2015) *ICT for sustainability: An emerging research field*. In: Hilty, L.M. and Aebischer, B. (eds.) *ICT Innovations for Sustainability*. Springer, pp.3–36.

Hook, M., Sovacool, B.K. and Sorrell, S. (2020) *A systematic review of the energy and climate impacts of teleworking*. *Environmental Research Letters*, 15(9), 093003.

10. Vihreän koodauksen tarkistuslistat

Nämä tarkistuslistat tukevat pohdintaa ja keskustelua katselmointien, suunnitteluarviointien tai retrospektiivien yhteydessä — ne eivät ole vaatimustenmukaisuustarkastuksia. Käytä niitä apuvälineinä energiaan liittyvien riskien tunnistamiseen, ei hyväksymis- tai hylkäämiskriteereinä.

10.1 Frontend – Web

- Onko uudelleenrenderöinti rajattu vain todella muuttuneisiin komponentteihin?
- Onko bundle-koko mitattu hiljattain, ja onko se perusteltu?
- Haetaan dataa vain tarvittaessa, eikä samaa sisältöä haeta useampaan kertaan?
- Tarjoillaanko kuvat ja media sopivassa resoluutiossa ja moderneissa formaateissa (WebP, AVIF)?
- Onko kolmannen osapuolen skriptien tarpeellisuus ja latausvaikutus arvioitu?
- Suositaanko CSS-siirtyimiä tai Web Animations API:a JavaScript-animaatiosilmukoiden sijaan?
- Onko polling- tai intervallipohjaiset mallit korvattu tapahtumaohjatulla tai push-pohjaisella lähestymistavalla, missä se on mahdollista?
- Onko HTTP-välimuisti määritetty oikein staattisille ja puolistaattisille resursseille?

10.2 Frontend – Mobiili

- Onko taustatehtävät minimoitu, konsolidoitu ja siirretty järjestelmän ajoittamiin ikkunoihin?
- Poistetaanko sensorien ja herätysvastaanotinten rekisteröinnit, kun niitä ei aktiivisesti tarvita?
- Tuetaanko offline-käyttäytymistä tarpeettomien verkon edestakaisin-pyyntöjen välttämiseksi?
- Niputetaanko verkkopyynnöt niin, että radio voi sammua aktiivisten jaksojen välillä?
- Onko renderöinti rajattu näkyvään sisältöön näkymäkierrätystä tai laiskoja listamalleja käyttäen?
- Onko paikannustarkkuus sovitettu todellisiin vaatimuksiin (karkea vs. tarkka)?
- Käytetäänkö push-ilmoituksia tilaan liittyvässä viestinnässä pollingin sijaan?
- Onko animaatiot ja siirtymät toteutettu laitteistokiihdytettyjä API:ja käyttäen?

10.3 Backend – Palvelut

- Onko pyyntökohtainen työ profiloitu, ja ovatko kuumat polut vapaita vältettävistä operaatioista?
- Ovatko tietokantakyselyt valikoivia, indeksoituja ja sivutettuja koko taulujen hakemisen sijaan?
- Onko N+1-kyselymallit poistettu innokkaalla latauksella tai niputetuilla kyselyillä?
- Onko hyötykuorman koko minimoitu, ja palautetaanko vain tarvittavat kentät?
- Ovatko välimuistit rajoitettuja, ja mitattiinko osumataajuuksia niiden muistikustannuksen oikeuttamiseksi?
- Onko kriittiseltä polulta siirretty pois synkroniset operaatiot, jotka voidaan turvallisesti lykätä?
- Ovatko joutilaiden yhteyksien, säiealtaiden ja suorittimien rajat asetettu resurssien loppumisen estämiseksi alhaisella kuormalla?
- Onko serialisointiformaatti sopiva käyttötapaukseen (esim. binäärimuodot sisäisille suuritehokkuuksille poluille)?

10.4 Dataputket

- Vältetäänkö muuttumattomien osioiden uudelleenlaskenta tarkistussummien, aikaleimamerkintöjen tai inkrementaalisten kehysten avulla?
- Käytetäänkö tarkistuspisteitä, jotta epäonnistuneet ajot voivat jatkua alusta aloittamisen sijaan?
- Onko käsittelytaajuus sovitettu todelliseen alajuoksun kulutusnopeukseen?
- Onko kylmä tai harvoin käytetty data siirretty vähäenergisempään tallennuskerrokseen?
- Onko datan säilyttämistä ohjattu määritellyllä elinkaari politiikalla, joka estää rajoittamattoman kasvun?
- Onko putken vaiheet profiloitu sen vaiheen tunnistamiseksi, joka hallitsee energiaa ja suoritusaikaa?

- Onko rinnakkaisuus sovitettu käytettävissä olevaan laitteistoon, välttämällä sekä ali- että yliprovisiointia?

10.5 Tekoälyn kouluttaminen

- Onko mallin kapasiteetti perusteltu tehtävän perusteella, vai täyttäisikö pienempi arkkitehtuuri vaatimukset?
- Onko kokeenseuranta käytössä kaksoiskappaleiden tai lähes identtisten koulutusajojen estämiseksi?
- Onko GPU/TPU-käyttöaste jatkuvasti korkea, vai onko datan lataus- tai esikäsittelypullon kauloja?
- Sovelletaanko varhaista pysäyttämistä, kun validointimittarit tasaantuvat?
- Käytetäänkö sekatalkkitarkeus- ja koulutusta (FP16/BF16), missä laitteisto ja kehys tukevat sitä?
- Onko aineistot suodatettu tai deduploitu mallilaatuun vaikuttamattomien näytteiden poistamiseksi?
- Onko hyperparametrien haut rajattu ja pysäytetään, kun vähenevät tuotot ovat ilmeisiä?

10.6 Tekoälyn päättely

- Käytetäänkö tuotannossa pienintä mallia, joka täyttää laatuvaatimukset?
- Ovatko syötteet ja kehoitteet mahdollisimman tiiviit, ja onko turha konteksti poistettu?
- Sovelletaanko välimuistia deterministisiin tai toistuviin kyselyihin tarpeettoman päättelyn välttämiseksi?
- Niputetaanko pyynnöt, missä viivevaatimukset sen sallivat?
- Onko mallin kvantisointia arvioitu pyyntökohtaisen laskennan vähentämiseksi?
- Seurattiinko päättelypisteen käyttöastetta, ja säädetäänkö kapasiteettia pitkittyneen joutokäynnin välttämiseksi?
- Onko upotuslaskelmat tallennettu välimuistiin sen sijaan, että ne lasketaan uudelleen jokaisella pyynnöllä?

11. Tulevaisuudennäkymät ja kehityssuunnat

Vihreä koodaus on kehittyvä tieteenala. Tässä oppaassa kuvatut periaatteet ja käytännöt edustavat näyttöön perustuvan ymmärryksen nykytilaa, mutta useita merkittäviä puutteita on edelleen — työkaluissa, koulutuksessa, organisatorisessa käytännössä ja tutkimuksessa — ja ne tulevat muokkaamaan alan kehitystä.

Paremmat mittaustyökalut. Useimmat olemassa olevat energiaprofiloijat on suunniteltu järjestelmätason tai prosessitasen analyysiin, eikä niillä ole integraatiota kehittäjien päivittäin käyttämiin työkaluihin. Kevyt, työnkulkuihin integroitu profilointi, joka yhdistää energiamittaukset versionhallinnan committeihin, konteistettuihin testiympäristöihin ja visuaalisiin koontinäyttöihin, on aktiivisesti kehittymässä oleva suunta. Prototyypitutkimus on osoittanut commit-tietoisien, toistettavan API-energiaprofiloinnin teknisen toteutettavuuden saavutettavalla laitteistolla, ja jatkotyö tähtää pilvipohjaisiin CI/CD-putkistoihin, usean profiloijan orkestrointiin ja hiili-intensiteettikartoitukseen (Joof et al., 2025; Joof, 2025).

Koulutus ja osaamisen kehittäminen. Kehittäjät eivät usein tiedosta, miten heidän suunnittelu- ja toteutuspäätöksensä vaikuttavat energiankulutukseen — ei siksi, että kiinnostusta puuttuisi, vaan koska energia ei ole kuulunut osaksi tavanomaista ohjelmistotekniikan koulutusta tai työkalujen palautesilmukoita. Energiatietoisuuden upottaminen opetussuunnitelmiin, koodikatselmoinnin käytäntöihin ja IDE-työkaluihin on välttämätöntä, jotta vihreä koodaus siirtyisi erityisasiantuntijoiden käytännöstä yleiseksi insinöörityön normiksi.

Organisatorinen yhtenäistäminen. Tekniset työkalut ovat tehokkaita vain silloin, kun organisaatiot kohtelevat energiatehokkuutta ensisijaisena laatutekijänä — suorituskyvyn, luotettavuuden ja ylläpidettävyyden rinnalla. Tämä edellyttää määriteltyjä kestävyystavoitteita, joita tukevat mittarit ja jotka on integroitu projektisuunnitteluun eikä kohdeltu valinnaisina tai jälkikäteinä.

Tutkimus–teollisuus-yhteistyö. Vihreän koodauksen merkittävimmät avoimet kysymykset — mukaan lukien se, miten energiaa voidaan mitata kohdistettavasti hajautetuissa ja pilvinatiiveissa järjestelmissä, miten koko järjestelmän laajuisista kompromisseista voidaan päätellä, ja miten vihreän koodauksen käytäntöjen pitkäaikainen tehokkuus voidaan validoida suuressa mittakaavassa — edellyttävät jatkuvaa yhteistyötä tutkijoiden ja käytännön ammattilaisten välillä. Aloitteet, jotka ankkueroivat tutkimuksen todellisiin kehitysympäristöihin ja tuottavat artefakteja, joita ammattilaiset voivat omaksua, ovat välttämättömiä akateemisten löydösten ja soveltavan käytännön välisen kuilun umpeen kuomiseksi.

Kohtelemalla energiajälkeä ensisijaisena ohjelmiston laatuattribuuttina vihreä koodaus mahdollistaa paremmat päätökset — ei täydellisyyttä, vaan jatkuvaa näyttöön perustuvaa parantamista. Tarvittavat työkalut, käytännöt ja organisatoriset olosuhteet tämän parantamisen ylläpitämiseksi ovat saavutettavissa, ja tämä opas on tarkoitettu panokseksi niiden kehittämiseen.

Viitteet

Joof, M.B. (2025) *Green Coding in Practice: A Software Framework for API Energy Efficiency Measurement and Feedback*. Master's thesis. Lappeenranta–Lahti University of Technology LUT.

Joof, M.B., Khan, M.A., Oyediji, S. and Porras, J. (2025) *Integrating API Energy Profiling into Developer Workflows: The VerdeFlow Prototype*. In Proceedings of the IEEE/ACM International Conference on Software Engineering: Workshop on Green and Sustainable Software (ICSE-GREENS). ACM.